



04

常用的ADB命令介绍



思考

为什么要学习命令呢？

- ✓ 效率高
- ✓ 面试会问
- ✓ 工具的本质也是调用命令
- ✓ 解决工具的能力边界

1、ADB工具介绍

- **ADB (Android Debug Bridge)** 是一个通用命令行工具，也是Android软件测试开发工作者常用的调试工具
- ADB可以用来**安装卸载软件、管理安卓系统软件、启动测试、抓取操作日志**等

2、ADB环境安装

- **下载安装包：**
 - 官网下载
 - 课堂已提供
- **安装步骤：**
 - 安装安卓SDK
 - 安装ADB
 - **配置环境变量（计算机右键—属性—高级系统设置—环境变量—新增系统变量）**

- **结果验证：**

```
C:\Users\18810>adb
Android Debug Bridge version 1.0.32
```

1.显示系统中全部设备：adb devices

```
C:\Users\18810>adb devices
adb server is out of date.  killing...
* daemon started successfully *
List of devices attached
emulator-5554    device
192.168.1.61:5555    device
```

2.开启或关闭ADB服务：

- 开启adb服务：adb start-server
- 关闭adb服务：adb kill-server

```
C:\Users\18810>adb start-server
* daemon not running. starting it now on port 5037 *
* daemon started successfully *

C:\Users\18810>adb kill-server

C:\Users\18810>_
```

3.连接/断开设备:

➤ 连接设置:

➤ 命令: **adb connect IP**

➤ 示例: adb connect 127.0.0.1:7555

```
Microsoft Windows [版本 10.0.19044.1586]  
(c) Microsoft Corporation. 保留所有权利。
```

```
C:\Users\Jack>adb devices 1. 检查当前连接信息  
List of devices attached
```

```
C:\Users\Jack>adb connect 127.0.0.1:7555 2. 连接指定的模拟器或手机 (如mumu)  
connected to 127.0.0.1:7555
```

```
C:\Users\Jack>adb devices 3. 确认设备已连接  
List of devices attached  
127.0.0.1:7555 device 设备已在线
```

```
C:\Users\Jack>adb disconnect 127.0.0.1:7555 4. 断开设备连接
```

```
C:\Users\Jack>adb devices 5. 确认设备已断开  
List of devices attached
```

➤ 断开设备:

➤ 命令: **adb disconnect IP**

➤ 示例: adb disconnect 127.0.0.1:7555

```
C:\Users\Jack>_
```

4. 安装/卸载软件包

➤ 安装软件:

- 命令: **adb install APK路径**
- 选项: [-r] 覆盖安装时保留数据和缓存文件。
- 示例: adb install soubaoShopMobile-release.apk

```
C:\soft\03功能测试\12app\04-app tools\apk>
```

```
C:\soft\03功能测试\12app\04-app tools\apk>adb devices
```

```
List of devices attached
```

```
127.0.0.1:7555 device
```

1. 确认设备连接信息

2. 安装tpshop

```
C:\soft\03功能测试\12app\04-app tools\apk>adb install soubaoShopMobile-release.apk
```

```
1982 KB/s (34077163 bytes in 16.782s)
```

```
pkg: /data/local/tmp/soubaoShopMobile-release.apk
```

```
Success
```

3. 卸载tpshop

```
C:\soft\03功能测试\12app\04-app tools\apk>adb uninstall com.tpshop.malls
```

```
Success
```

➤ 卸载软件:

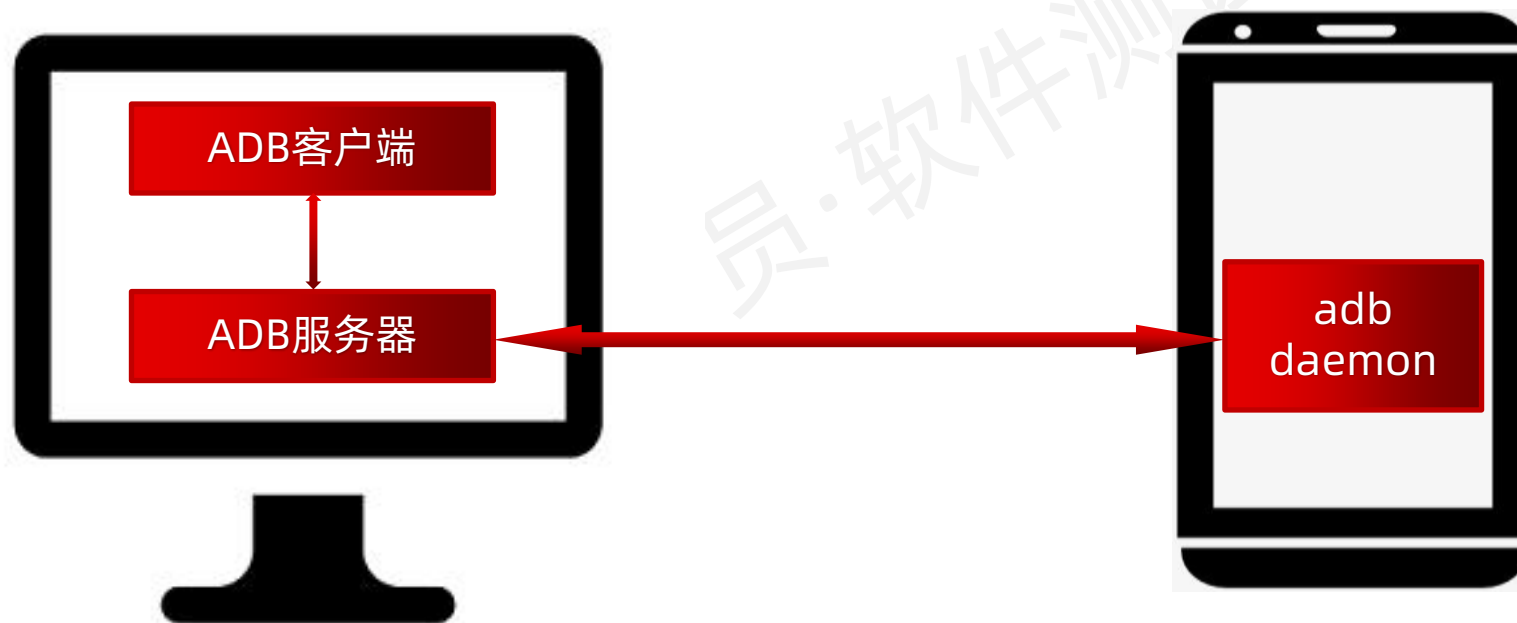
```
C:\soft\03功能测试\12app\04-app tools\apk>_
```

- 命令: **adb uninstall apk包名**
- 选项: [-k] 卸载时保留数据和缓存文件
- 示例: adb uninstall com.tpshop.malls

adb工作原理

ADB

说明： ADB 全名 Android Debug Bridge，是一个手机调试工具



- **Client端**：运行在开发机器中，即你的开发电脑，用来发送 adb 命令；
- **Server端**：同样运行在开发机器中，用来管理 Client 端和手机的 Daemon 之间的通信。

- **Daemon守护进程**：运行在调试设备中，手机或模拟器，用来接收并执行 adb 命令；

5. 获取软件包名

- 列出手机装的所有app的包名: `adb shell pm list packages`
- 列出系统应用的所有包名: `adb shell pm list packages -s`
- 列出除了系统应用的**第三方**应用包名: `adb shell pm list packages -3`
- 显示当前打开的软件包名:
 - **Windows**: `adb shell dumpsys window | findstr mCurrentFocus`
 - **Mac/Linux**: `adb shell dumpsys window | grep mCurrentFocus`

概念

- **包名 (package)**: 决定程序的唯一性 (不是应用的名字)
- **界面名 (activity)**: 目前可以理解, 一个界面名, 对应着一个界面

- 注意: 需要打开目标软件, 如tpshop

```
C:\soft\03功能测试\12app\04-app tools\apk>adb shell pm list packages -3
package:com.alipay.hulu
package:com.tpshop.malls

C:\soft\03功能测试\12app\04-app tools\apk>adb shell dumpsys window | findstr mCurrentFocus
mCurrentFocus=Window{aa2fbfd u0 com.tpshop.malls/com.tpshop.malls.WelcomeActivity}

C:\soft\03功能测试\12app\04-app tools\apk>
```


6.清除应用数据与缓存

- 命令: **adb shell pm clear (apk包名)**
- 示例: `adb shell pm clear com.tpshop.malls`

```
C:\soft\03功能测试\12app\04-app tools\apk>adb shell pm clear com.tpshop.malls
Success
```

7.启动、停止应用

➤ 启动:

- 命令: **adb shell am start 包名/Activity名**
- 示例: `adb shell am start com.tpshop.malls/com.tpshop.malls.SplashActivity`

➤ 停止:

- 命令: **adb shell am force-stop (apk包名)**
- 示例: `adb shell am force-stop com.tpshop.malls`

```
C:\soft\03功能测试\12app\04-app tools\apk>adb shell am start com.tpshop.malls/com.tpshop.malls.SplashActivity
Starting: Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.tpshop.malls/.SplashActivity }

C:\soft\03功能测试\12app\04-app tools\apk>adb shell am force-stop com.tpshop.malls

C:\soft\03功能测试\12app\04-app tools\apk>_
```

adb常用命令 - 获取手机日志



测试过程中发现bug,
获取日志信息



获取错误日志步骤

当测试过程中发现问题后想获取错误日志信息：

- 打开被测应用程序，进入到触发缺陷的位置
- 使用查看日志命令：**adb logcat**
- 触发缺陷
- 获取日志信息

```
E/AndroidRuntime( 6593): FATAL EXCEPTION: main
E/AndroidRuntime( 6593): Process: cn.itcast.myapplication, PID: 6593
E/AndroidRuntime( 6593): java.lang.ArrayIndexOutOfBoundsException: length=5; index=8
E/AndroidRuntime( 6593):     at cn.itcast.myapplication.LoginActivity.attemptLogin(LoginActivity.java:149)
E/AndroidRuntime( 6593):     at cn.itcast.myapplication.LoginActivity.access$000(LoginActivity.java:40)
E/AndroidRuntime( 6593):     at cn.itcast.myapplication.LoginActivity$2.onClick(LoginActivity.java:89)
E/AndroidRuntime( 6593):     at android.view.View.performClick(View.java:4780)
E/AndroidRuntime( 6593):     at android.view.View$PerformClick.run(View.java:19866)
E/AndroidRuntime( 6593):     at android.os.Handler.handleCallback(Handler.java:739)
E/AndroidRuntime( 6593):     at android.os.Handler.dispatchMessage(Handler.java:95)
E/AndroidRuntime( 6593):     at android.os.Looper.loop(Looper.java:135)
E/AndroidRuntime( 6593):     at android.app.ActivityThread.main(ActivityThread.java:5254)
E/AndroidRuntime( 6593):     at java.lang.reflect.Method.invoke(Native Method)
E/AndroidRuntime( 6593):     at java.lang.reflect.Method.invoke(Method.java:372)
E/AndroidRuntime( 6593):     at com.android.internal.os.ZygoteInit$MethodAndArgsCaller.run(ZygoteInit.java:903)
E/AndroidRuntime( 6593):     at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:698)
```

注意

一般的情况下留意Error错误级别的日志。另外类似如图中的一些异常错误日志信息。在触发错误日志时及时ctrl+c掐断日志刷新。

8. 获取APP日志

➤ 命令: **adb logcat** > 指定路径

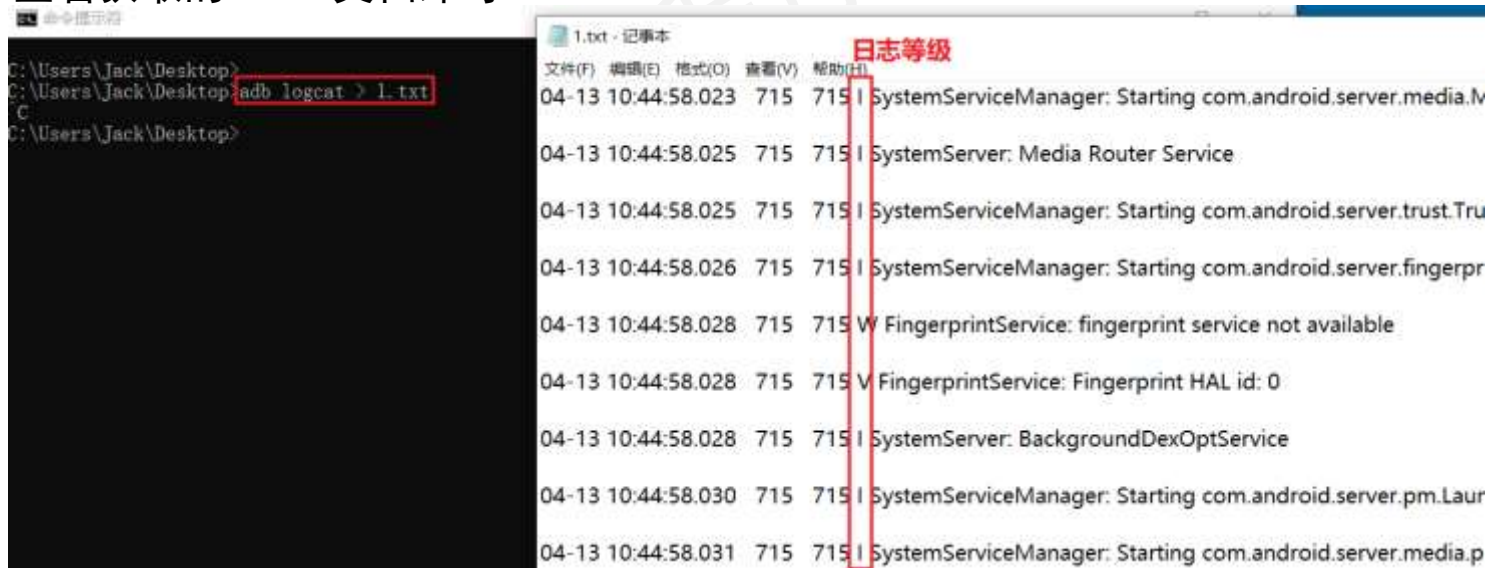
➤ 案例: 抓取APP日志信息并保存到1.txt文件中

① **adb logcat** >C:\Users\18810\Desktop\1.txt

② 执行完后Ctrl+C结束日志获取

③ 查看获取的1.txt文档即可

```
V Verbose (default for <tag>)
D Debug (default for '*')
I Info
W Warn
E Error
F Fatal
S Silent (suppress all output)
```



The screenshot shows a terminal window on the left with the command `adb logcat > 1.txt` entered. On the right, a text file named '1.txt' is open, displaying the captured log output. A red box highlights the first column of the log entries, which are labeled '日志等级' (Log Level) in the header. The log entries include timestamps, process IDs, and log levels (I for Info, W for Warn, V for Verbose).

日志等级	Timestamp	Process ID	Log Message
I	04-13 10:44:58.023	715	SystemServiceManager: Starting com.android.server.media.l
I	04-13 10:44:58.025	715	SystemServiceManager: Starting com.android.server.tru
I	04-13 10:44:58.026	715	SystemServiceManager: Starting com.android.server.fingerpr
W	04-13 10:44:58.028	715	FingerprintService: fingerprint service not available
V	04-13 10:44:58.028	715	FingerprintService: Fingerprint HAL id: 0
I	04-13 10:44:58.028	715	SystemServiceManager: Starting com.android.server.pm.Laur
I	04-13 10:44:58.031	715	SystemServiceManager: Starting com.android.server.media.p

adb常用命令 - 文件传输



上传文件

adb push 电脑的文件路径 手机的文件夹路径

小练习

将桌面的 a.txt 发送到手机的 sd 卡

下载文件

adb pull 手机的文件路径 电脑的文件夹路径

小练习

将手机的 sd 卡的 a.txt 拉取到桌面

```
C:\Users\Jack>adb push Desktop/1.txt /sdcard
Desktop/1.txt: 1 file pushed, 0 skipped.
```

```
C:\Users\Jack>adb pull /sdcard/1.txt Desktop/2.txt
/sdcard/1.txt: 1 file pulled, 0 skipped. 0.0 MB/s (5 bytes in 0.002s)
```

adb常用命令 - app启动时间



某微打开应用 1s, 去支付。
某宝打开应用 10s, 去支付。



测试人员对于APP的启动速度必须进行测试！！

测试通过标准

- 需求有明确的启动时间指标
- 参考同类软件，启动时间不能大于竞争对手的启动时间

启动app

```
adb shell am start -W 包名/启动名
```

结果说明

ThisTime : 该界面 (activity) 启动耗时 (毫秒)

TotalTime : 应用自身启动耗时 = ThisTime + 应用 application 等资源启动时间 (毫秒)

WaitTime : 系统启动应用耗时 = TotalTime + 系统资源启动时间 (毫秒)

WaitTime

TotalTime

Thistime

系统

Application

Activity

9. 获取APP启动时间

➤ 格式: **adb shell am start -W 包名/activity名**

➤ 常见参数:

-S: 表示每次启动前先强行停止

-R: 表示重复测试次数

➤ 常见的三个指标

➤ ThisTime: 当前activity的时间。

➤ **TotalTime: 应用的启动时间, 包括创建进程、App初始化、Activity初始化到界面显示。**

➤ WaitTime: 前一个应用activity pause的时间+TotalTime

➤ 举例: **adb shell am start -W -S -R 3 com.tpshop.malls/com.tpshop.malls.SplashActivity**

```
C:\Users\Jack>
C:\Users\Jack>adb shell am start -W com.tpshop.malls/com.tpshop.malls.SplashActivity
Starting: Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.tpshop.malls/.SplashActivity }
Status: ok
Activity: com.tpshop.malls/.SplashActivity
ThisTime: 484
TotalTime: 484 冷启动
WaitTime: 499
Complete

C:\Users\Jack>adb shell am start -W com.tpshop.malls/com.tpshop.malls.SplashActivity
Starting: Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.tpshop.malls/.SplashActivity }
Warning: Activity not started, its current task has been brought to the front
Status: ok
Activity: com.tpshop.malls/.SPMainActivity
ThisTime: 0
TotalTime: 0 热启动
WaitTime: 3
Complete

C:\Users\Jack>
```


10. 获取内存

- 格式: **adb shell dumpsys meminfo <包名>**
- 示例: **adb shell dumpsys meminfo com.tpshop.malls**
- 检查点:

- **Native/Dalvik 的 Heap 信息**

- 如果发现这个值一直增长, 则代表程序可能出现了内存泄漏 (Out of memory)。

- **Total 的 PSS 信息**

- 这个值是应用真正占据的内存大小, 通过这个信息, 可以轻松判别手机中哪些程序占内存比较大

```
~\Users\jack\Desktop>adb shell dumpsys meminfo com.tpshop.malls
Applications Memory Usage (kbytes):
  uptime: 12675914 Realtime: 12675914

** MEMINFO in pid 4302 [com.tpshop.malls] **
    Pss Private Private  Swapped   Heap   Heap   Heap
    Total Dirty Clean   Dirty   Size   Alloc   Free
Native Heap      0      0      0      0 26368 19223  7144
Dalvik Heap  34250 34196      0      0 46491 33981 12510
Dalvik Other  2083  2080      0      0
  Stack        80      0      0      0
  Ashmem       2      0      0      0
  Other dev     4      0      4      0
  .so mmap    8023  516 1444      0
  .apk mmap    746      0  272      0
  .ttf mmap    252      0  252      0
  .dex mmap   4746      4 1536      0
  .oat mmap  10633      0 2753      0
  .art mmap   2623 1056      56      0
  Other mmap   6127      0 5960      0
  Unknown    17479 17256      0      0
  TOTAL    37047 55196 12276      0 72859 53204 19654

App Summary
      Pss(KB)
-----
Java Heap:  35308
Native Heap:    0
Code:       6776
```

```
App Summary
      Pss(KB)
-----
Java Heap:  38912
Native Heap:    0
Code:       9864
Stack:       80
Graphics:    0
Private Other: 24188
System:     11830
TOTAL:  84874  TOTAL SWAP (KB):  0

Objects
Views: 874      ViewRootImpl: 1
AppContexts: 4      Activities: 3
Assets: 2      AssetManagers: 2
Local Binders: 17      Proxy Binders: 25
Parcel memory: 7      Parcel count: 30
Death Recipients: 2      OpenSSL Sockets: 0

SQL
MEMORY_USED: 0
PAGECACHE_OVERFLOW: 0      MALLOC_SIZE: 0
```

11. 查看CPU占用情况

➤ 格式: **adb shell top -s 列号**

➤ 说明: [-s] 按指定行排序

➤ 参数含义:

- PID : 进程ID
- USER: 进程所有者用户名
- PR: 优先级
- NI: nice值
- VIRT: 进程使用的虚拟内存总量
- RES: 进程实际使用内存
- SHR: 共享内存大小
- S : 进程的状态
- **%CPU: 进程所占用的CPU百分比**
- **%MEM: 进程所占用的物理内存百分比**
- TIME+: 进程使用的CPU时间总和
- ARGS: 程序

```
C:\Users\Jack>adb shell top
top - 7:25:11 AM, tasks: 79 total, 1 running, 78 sleeping, 0 stopped, 0 zombie
Mem: 9200208k total, 508132k used, 8692076k free, 1912k buffers
Swap: 0k total, 0k used, 0k free, 151188k cached
400%cpu 0%user 0%nice 4%sys 396%idle 0%low 0%irq 0%irq 0%host
PID USER PR NI VIRT RES SHR S [CPU] MEM TIME+ ARGS
2163 root 20 0 6.2M 2.5M 2.1M R 0.0 0.0 0:00.00 top
1893 u0_a48 20 0 817M 160M 77M S 0.0 1.7 0:44.16 com.tpshop.malls
1576 u0_a48 20 0 794M 79M 54M S 0.0 0.8 0:00.63 com.tpshop.malls:pushcore
1295 u0_a1 20 0 745M 46M 31M S 0.0 0.5 0:00.06 com.android.providers.calendar
1273 system 20 0 733M 41M 26M S 0.0 0.4 0:00.02 com.android.keychain
1009 u0_a2 20 0 749M 52M 35M S 0.0 0.5 0:00.12 android.process.acore
902 u0_a4 20 0 749M 50M 33M S 0.0 0.5 0:00.09 android.process.media
898 wifi 20 0 10M 3.0M 2.4M S 0.0 0.0 0:00.05 wpa_supplicant -c/data/misc/wifi/wpa_supplicant.conf -i wlan0 -Dwired -O/data/misc/wifi/sockets -e/da
800 u0_a10 20 0 790M 84M 50M S 0.0 0.9 0:01.25 com.android.systemui
716 system 18 -2 1.6G 95M 55M S 0.0 1.0 0:14.78 system_server
345 root 20 0 6.7M 424K 0 S 0.0 0.0 0:00.01 nemuVM-nemu-service
330 root 20 0 3.5M 304K 188K S 0.0 0.0 0:00.00 mu_bak --daemon
326 root 20 0 7.4M 828K 492K S 0.0 0.0 0:00.03 opengl_gc
324 root 20 0 5.3M 832K 620K S 0.0 0.0 0:00.00 powerbtnd
323 system 20 0 9.1M 2.5M 1.9M S 0.0 0.0 0:00.04 gatekeeperd /data/misc/gatekeeper
322 root 20 0 1.4G 10M 0 S 0.0 0.1 0:00.66 zygote
321 root 20 0 1.4G 39M 25M S 0.0 0.4 0:00.78 zygote64
319 keystore 20 0 10M 2.2M 1.6M S 0.0 0.0 0:00.00 keystore /data/misc/keystore
318 root 20 0 5.6M 556K 304K S 0.0 0.0 0:00.00 install
317 media 20 0 95M 4.6M 0 S 0.0 0.0 0:00.27 mediaserver
316 drm 20 0 16M 956K 56K S 0.0 0.0 0:00.02 drmserver
315 root 20 0 6.4M 1.7M 1.2M S 0.0 0.0 0:00.00 rild
314 root 20 0 5.9M 900K 644K S 0.0 0.0 0:00.00 debuggerd64
313 root 20 0 4.1M 228K 0 S 0.0 0.0 0:00.00 debuggerd
312 root 20 0 17M 1.9M 1.2M S 0.0 0.0 0:00.23 netd
303 root 20 0 8.7M 708K 492K S 0.0 0.0 0:00.00 local_camera_back
302 root 20 0 5.3M 1.4M 1.0M S 0.0 0.0 0:01.64 local_gps
301 root 20 0 5.3M 800K 588K S 0.0 0.0 0:00.00 vinput
300 root 20 0 5.2M 776K 576K S 0.0 0.0 0:00.00 ntp_server
299 root 12 -8 32M 3.4M 2.0M S 0.0 0.0 1:04.39 surfaceflinger
298 system 20 0 5.9M 776K 544K S 0.0 0.0 0:00.03 servicemanager
297 root -2 0 5.8M 836K 568K S 0.0 0.0 0:00.00 lmkd
296 root 20 0 7.3M 0.9M 752K S 0.0 0.0 0:00.01 adbd --root_seclabel=u:r:su:s0
295 root 20 0 4.3M 876K 700K S 0.0 0.0 0:00.01 healthd
```


12. 获取APP使用流量

➤ 获取进程pid

➤ win: adb shell ps | findstr 包名

➤ mac: adb shell ps | grep 包名

➤ 如: adb shell ps | findstr com.tpshop.malls

➤ 获取流量

➤ adb shell cat /proc/{pid}/net/dev

➤ 查看

➤ Wlan0: wifi网卡

➤ Receive是接受、Transmit是发送

```
C:\Users\Jack>
C:\Users\Jack>
C:\Users\Jack>adb shell ps | findstr com.tpshop.malls 1、获取程序PID
u0_a48    1576   321   812728 81496      0 7fab85a25c3a S com.tpshop.malls:pushcore
u0_a48    1893   321   842956 158160     0 7fab85a25c3a S com.tpshop.malls

C:\Users\Jack>adb shell cat /proc/1893/net/dev 2.获取流量
Inter-|   Receive   |   Transmit   |
face  | bytes  packets | bytes  packets |
wlan0: 915212  3563   746473  3516
ifb0:   0      0     0      0
ifb1:   0      0     0      0
sit0:   0      0     0      0
lo: 11433021 128826 0      0 11433021 128826
```

性能稳定性测试介绍

稳定性测试：通过长时间对应用程序进行无序操作，检验应用程序是否会出现异常。如闪退crash、无响应ANR等。

稳定性测试工具——Monkey：

- Monkey是一个命令行工具，是由安卓官方提供的。
- 测试人员可以通过Monkey来模拟用户的**触摸、点击、滑动**以及**系统按键**等操作（操作事件都是随机的），从而实现对APP压力的测试和稳定性测试。
- 开发人员结合monkey 打印的日志和系统打印的日志，修改测试中出现的问题。

稳定性测试的时机：

- 一般需要等产品稳定了，bug比较少的时候，再用monkey去测试待测试应用的稳定性。

Monkey测试

➤ 语法: **adb shell monkey -p 包名 -v 次数 > c:\日志.txt**

➤ 说明:

➤ -p 指定包名

➤ -v 日志详细程度 (最高支持' -v -v -v' 最详细)

➤ 示例: **adb shell monkey -p com.tpshop.malls -v 20000 > C:\Users\Jack\Desktop\monkey.txt**

```
monkey.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
:Monkey: seed=1650130222972 count=1000

:AllowPackage: com.tpshop.malls

:IncludeCategory: android.intent.category.LAUNCHER
:IncludeCategory: android.intent.category.MONKEY

// Event percentages:
// 0: 15.0%
// 1: 10.0%
// 2: 2.0%
// 3: 15.0%
// 4: -0.0%
// 5: -0.0%
// 6: 25.0%
```

```
C:\Users\Jack>adb shell monkey -p com.tpshop.malls -v 1000 >adb shell monkey -p com.tpshop.malls -v 10000 > C:\Users\Jack\Desktop\monkey.txt
C:\Users\Jack>
```

分类	参数	释义
基础参数	-v	指定反馈信息级别（信息级别就是日志的详细程度），总共分3个级别。最高级别为-v -v -v
	-s	指定monkey测试的序列
	-p	指定一个或多个包（Package，即App）
	--throttle	指定用户操作（即事件）间的时延，单位是毫秒
事件类型	--pct-touch	调整触摸事件的百分比(触摸事件是一个down-up事件，它发生在屏幕上的某单一位置)
	--pct-motion	调整动作事件的百分比(动作事件由屏幕上某处的一个down事件、一系列的伪随机事件和一个up事件组成)
	--pct-trackball	调整轨迹事件的百分比(轨迹事件由一个或几个随机的移动组成，有时还伴随有点击)
	--pct-syskeys	调整“系统”按键事件的百分比(这些按键通常被保留，由系统使用，如Home、Back、Start Call、End Call及音量控制键)
	--pct-anyevent	调整其它类型事件的百分比。它包罗了所有其它类型的事件，如：按键、其它不常用的设备按钮、等等
调试选项	--ignore-crashes	应用程序崩溃
	--ignore-timeouts	无响应ANR
	--igone-security-exceptions	许可证书崩溃
	--kill-process-after-error	发生错误停止运行并保持当前状态
	--montitor-native-crashes	监视并报告Androids系统本地代码的崩溃事件



总结

1. 启动时间:adb shell am start -W 包名/activity名
2. 内存: adb shell dumpsys meminfo <包名>
3. CPU: adb shell **top**
4. 安装: adb **install** 软件包路径
5. 卸载: adb **uninstall** 包名
6. 抓取日志:adb **logcat** >指定路径
7. **Monkey**: adb shell monkey -p 包名 -v 次数 >c:\日志.txt
8. 流量:
 - ✓ 获取**pid**:
 - ✓ windows: adb shell ps | findstr 包名
 - ✓ mac: adb shell ps | grep 包名
 - ✓ 查看流量: adb shell cat /proc/{**pid**}/net/dev



05

软件开发方法及应用发布



思考

互联网应用（京东）与传统行业应用（个税APP)更新速度一样吗？

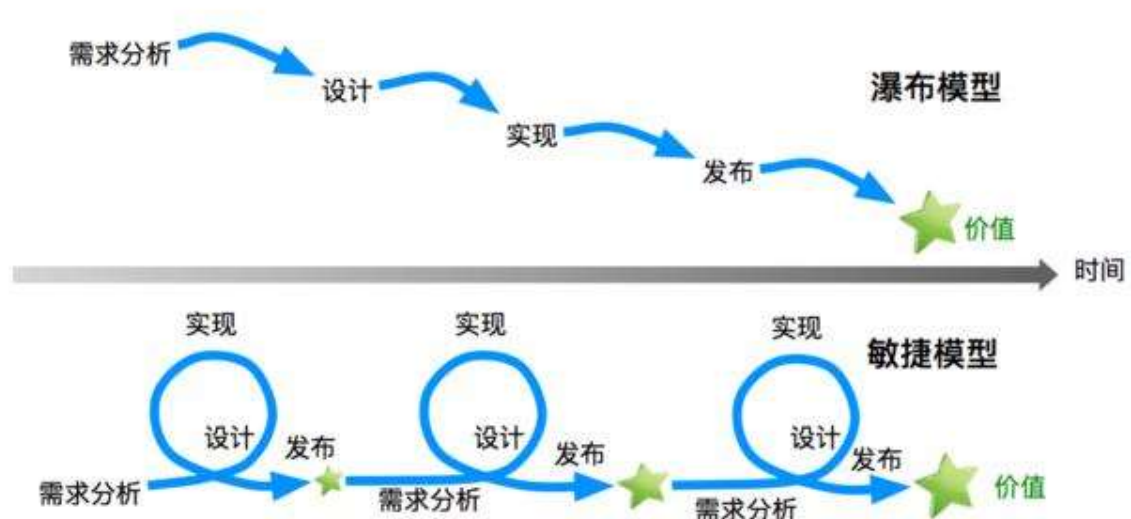
软件开发方法

迭代速度不同：开发模型不一样

传统行业：瀑布模型

互联网+：敏捷模型

开发模型



瀑布模型：将一个项目作为一个整体，下一个环节依赖上一个环节的完成。

敏捷模型：将一个项目拆分成多个子项目，每一个迭代周期完成一个子项目

敏捷开发（scrum）模型

Scrum：是一个敏捷开发框架，是一个**增量**的，**迭代**的开发过程

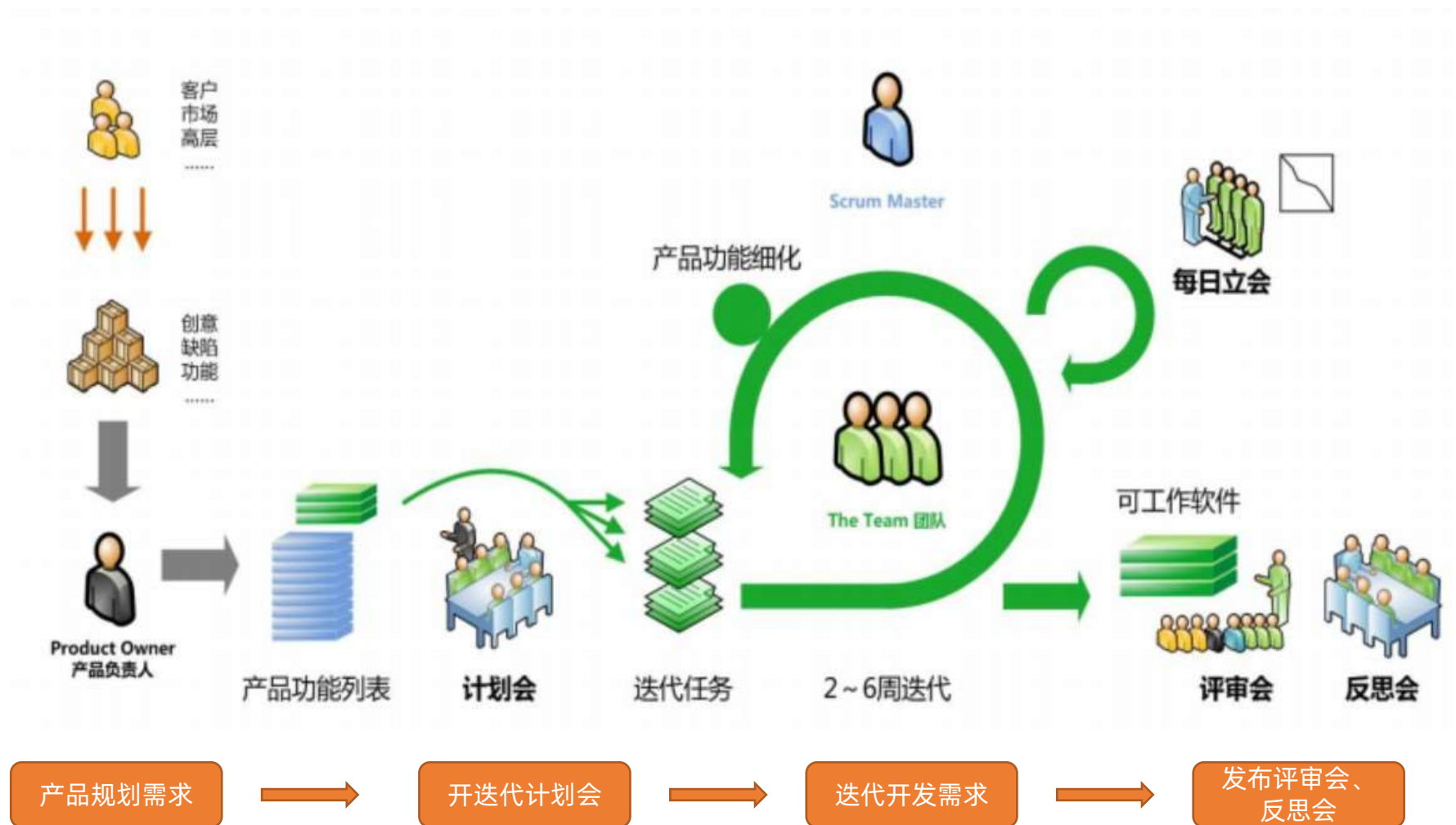
- **迭代（sprint）**：项目开发过程中最小周期，每个sprint周期建议为**2周**。在scrum框架中，整个开发周期包括**若干个小的迭代周期**。
- 产品功能列表（Backlog）：在Scrum中，将产品Backlog按**商业价值**排出需求列表



Scrum三种角色：

- Product Owner(**产品**负责人)
 - 定义需求，进行需求排期
- Scrum Master(**项目**经理)
 - 管理项目，确保scrum顺利执行
- Dev Team(**开发团队**)
 - 实现客户需求
 - 成员：**开发、测试、UI**。
 - 团队人数：一般**5人到9人**。开发测试比一般为：**3:1 — 5:1**

敏捷开发模式 —— scrum敏捷模型





总结

1、什么是敏捷模型？

- 核心是**将一个项目拆分成多个子项目**，**每一个迭代周期完成一个子项目**。

2、敏捷模型Scrum

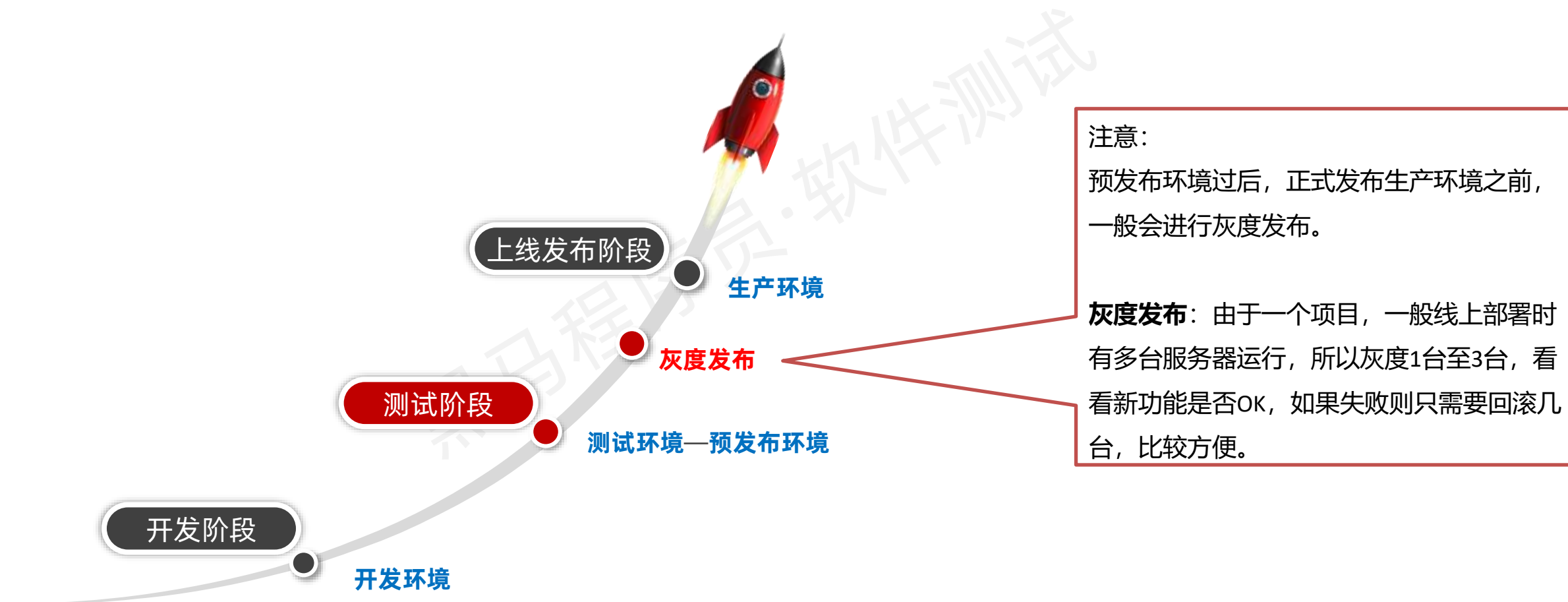
- 三种角色：
 - 产品负责人（Product Owner）
 - 项目经理（Scrum Master）
 - 开发团队（Dev Team）：包含**开发、测试、UI**。
- 迭代周期：**2周**左右



思考

100台服务器，发布新版本时，1次性更新100台服务器好，还是先更新几台验证一段时间好？

项目上线发布策略



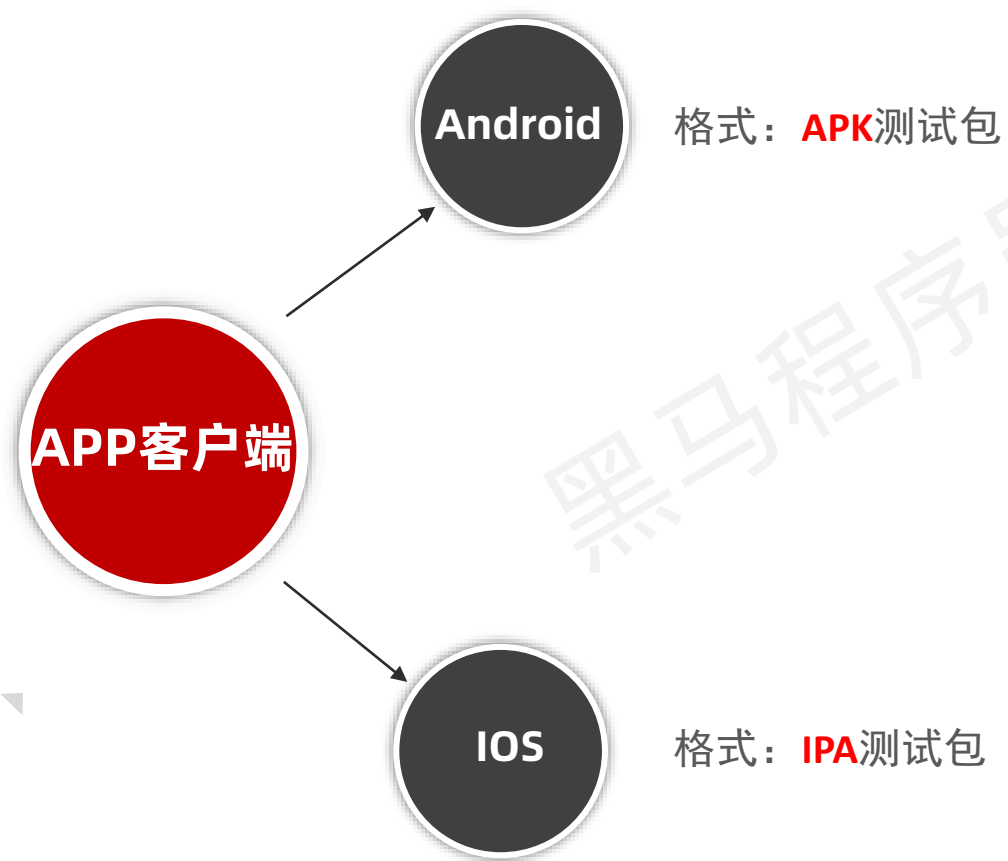


思考

APP如何发布?

APP软件包类型

- APP开发完成后，相应的**开发人员会打出应用程序包**，由测试人员安装测试



注意

IOS APP和Android APP在**界面上的功能一样**，但**实际上是两个完全独立的项目**。

- 使用**不同的开发语言**
- 由**不同的项目组成员**进行开发

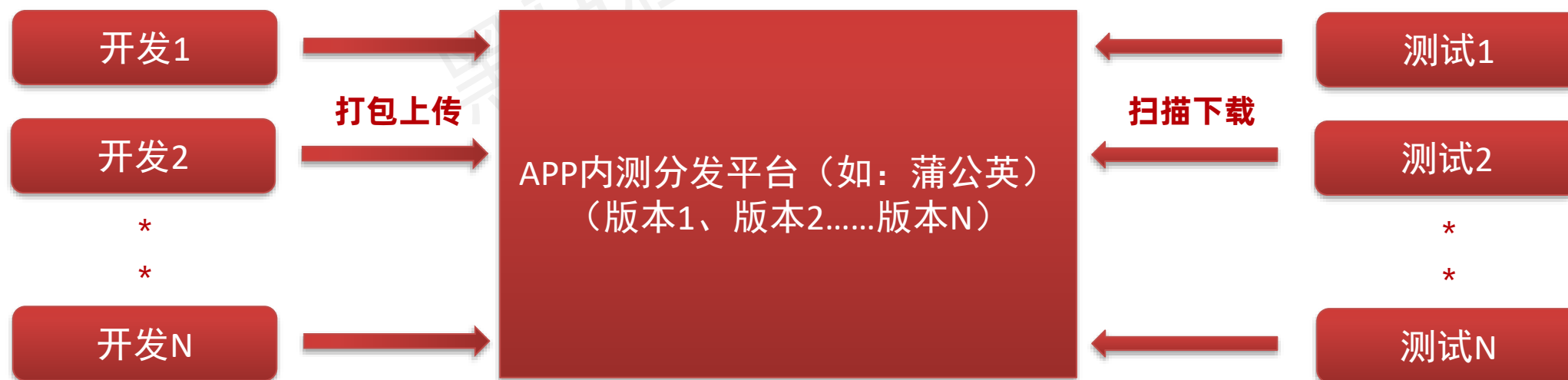


思考

APP测试包如何发布和管理?

APP客户端（内部）发布平台

- 在实际测试工作中，为了方便测试程序包的安装和管理，可以使用一些应用**内测分发平台**。如：**蒲公英**、**Testlink**等
- 操作步骤：
 - 开发将应用测试包**上传**到这些平台上
 - 平台可以**生成对应的二维码**
 - **测试直接扫码进行应用安装**





思考

APP包如何发布给用户?

APP客户端（线上）发布平台

- 产品测试完成后要在线上发布，让用户进行下载使用。下面是安卓和IOS应用常用的发布平台和渠道
 - **Android应用**：豌豆荚、应用宝、360手机助手、各类手机品牌商城等；
 - **IOS应用**：主要有 App store、iTools
- 操作步骤：
 - **开发者账号注册**，申请在发布平台（各种应用商店）上架
 - 针对不同的发布平台，在软件包中加入对应的平台ID（渠道ID），**上传**到发布平台
 - **平台审核**通过后，用户即可在应用商店中下载

注意事项

- 一般线上发布过程，由**开发人员**负责。
- 在软件包加入平台ID后，上传到发布平台时，需要**测试人员**验证**核心的业务功能**

总结

1、互联网公司使用什么开发模型？多久迭代一次？

- ✓ 敏捷开发模型

- ✓ **2周**

2、什么是灰度发布？

- ✓ 时间：预发布环境测试之后，生产环境发布之前

- ✓ 策略：**部分用户可用，若有异常则回滚**



传智教育旗下高端IT教育品牌